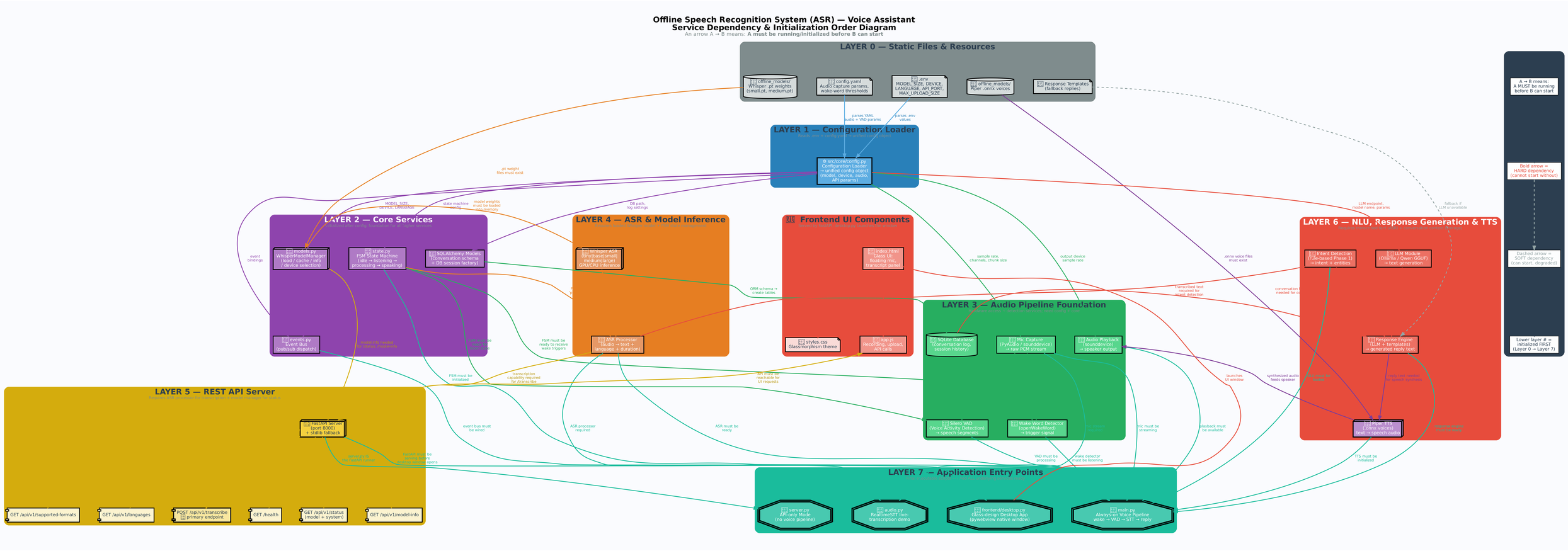

# Offline Speech Recognition System (ASR) — Voice Assistant
# Service Dependency & Initialization Order Diagram

An arrow A → B means: **A must be running/initialized before B can start**

LAYER 0 — Static Files & Resources
offline_models/ Whisper .pt weights (small.pt, medium.pt)
config.yaml Audio capture params, wake-word thresholds
.env MODEL_SIZE, DEVICE, LANGUAGE, API_PORT, MAX_UPLOAD_SIZE
offline_models/ Piper .onnx voices
Response Templates (fallback replies)
parses YAML audio + VAD params
parses .env values
LAYER 1 — Configuration Loader
Reads .env + config.yaml → unified config object
⚙ src/core/config.py Configuration Loader → unified config object (model, device, audio, API params)
.pt weight files must exist
MODEL_SIZE, DEVICE, LANGUAGE
state machine config
model weights must be loaded into memory
DB path, log settings
event bindings
LAYER 2 — Core Services
Initialized after config; foundation for all higher services
models.py WhisperModelManager (load / cache / info / device selection)
state.py FSM State Machine (idle → listening → processing → speaking)
SQLAlchemy Models (conversation schema + DB session factory)
events.py Event Bus (pub/sub dispatch)
model info needed for /status, /model-info
FSM must be ready for listen state
FSM must be ready to receive wake triggers
LAYER 4 — ASR & Model Inference
Requires loaded Whisper model + FSM state management
Whisper ASR (tiny|base|small|medium|large) GPU/CPU inference
ASR Processor (audio → text + language + duration)
ORM schema → create tables
Frontend UI Components
Served by FastAPI; desktop.py launches the window
index.html Glass UI: floating mic, transcript panel
styles.css Glassmorphism theme
app.js Recording, upload, API calls
sample rate, channels, chunk size
output device sample rate
LAYER 3 — Audio Pipeline Foundation
Hardware access → detection services; need config + core
SQLite Database (conversation log, session history)
Mic Capture (PyAudio / sounddevice) → raw PCM stream
Audio Playback (sounddevice) → speaker output
Silero VAD (Voice Activity Detection) → speech segments
Wake Word Detector (openWakeWord) → trigger signal
LLM endpoint, model name, params
fallback if LLM unavailable
.onnx voice files must exist
transcribed text required for intent detection
conversation needed for c
LAYER 6 — NLU, Response Generation & TTS
Requires transcribed text (ASR) + conversation context (storage)
Intent Detection (rule-based Phase 1) → intent + entities
LLM Module (Ollama / Qwen GGUF) → text generation
Response Engine (LLM + templates) → generated reply text
Piper TTS (.onnx voices) text → speech audio
launches UI window
synthesized audio feeds speaker
NLU must be loaded
reply text needed for speech synthesis
response engine must be ready
TTS must be initialized
transcription capability required for /transcribe
FSM must be initialized
API must be reachable for UI requests
event bus must be wired
ASR processor required
ASR must be ready
mic stream required
mic must be streaming
playback must be available
VAD must be processing
wake detector must be listening
LAYER 5 — REST API Server
Requires ASR processor for transcription + model manager for status
FastAPI Server (port 8000) + stdlib fallback
server.py IS the FastAPI runner
FastAPI must be serving before desktop window opens
GET /api/v1/supported-formats
GET /api/v1/languages
POST /api/v1/transcribe primary endpoint
GET /health
GET /api/v1/status (model + system)
GET /api/v1/model-info
LAYER 7 — Application Entry Points
Final executable scripts — need ALL underlying services ready
server.py API-only Mode (no voice pipeline)
audio.py RealtimeSTT live-transcription demo
frontend/desktop.py Glass-design Desktop App (pywebview native window)
main.py Always-on Voice Pipeline wake → VAD → STT → reply
A → B means: A MUST be running before B can start
Bold arrow = HARD dependency (cannot start without)
Dashed arrow = SOFT dependency (can start, degraded)
Lower layer # = initialized FIRST (Layer 0 → Layer 7)